

Software Engineering :

7 : Software Architecture

Part 1

Outline :

- **Part 1:**
 - **Introduction to Software Architecture**
 - **Software Architecture Process**
 - **Selecting Technological Stack**
 - **Architecture Quality Attributes**
 - **Software Architectural Styles**
- **Part 2 :**
 - **Programming Questions**
 - **Design Patterns**
 - **Architecture Principles : SOLID**
- **Part 3 :**
 - **Decoupling & Cohesion**
 - **Documenting the Architecture**

Introduction to Software Architecture

- ***What's Software Architecture :***

- It is the process to design and define the functional structure of a software system via:
 - Identifying the different components, elements and subsystems composing the whole software.
 - Defining the behaviour of the software components in addition to their interfaces.
- It is a process focused on the structure not on the code and implementation details.

Introduction to Software Architecture

- ***What's Software Architecture :***

- It is the link between the requirements engineering phase and design phase where specifications are analysed and turned into a document designing the high-level structuring and behaviour of the system in order to produce low-level designs (Including UI, database, class diagram...)

Software Architecture

Fundamental structures of a system.....

which comprise:

- software elements
- relations among them
- properties of both.

Software Architecture

Architecture vs Design

The distinction is not always clear-cut

Architecture focuses more on: High level structure of a software system

Significant design decisions of a system that

if you have to change them => High cost

"All architecture is design but not all design is architecture" G. Booch

Software Architecture

Architecture = high-level design

The focus shifts from small units (e.g., classes) to larger and more relevant units, such as packages, modules, subsystems, layers, and services

Introduction to Software Architecture

- ***Can we develop the To-Do App without Architectural Design :***
 - Yes , we can start developing directly or using some framework
 - We conduct some UML modeling for the class, use case, sequence ...(But this is not architectural design)
 - **But : ?**

Introduction to Software Architecture

- ***Can we develop the To-Do App without Architectural Design :***
 - Yes , we can start developing directly or using some framework
 - We conduct some UML modeling for the class, use case, sequence ...(But this is not architectural design)
 - **But : at what cost for the following questions ?**
 - Can it Scale to support a large number of users ?
 - What happens if change the database engine to Oracle ? NoSQL ?
 - Can we integrate a native mobile application ?
 - Integrate other authentication-services including Google (OAuth..) ?

Introduction to Software Architecture

- ***Why Software Architecture is Important:***
 - Provides a representation that facilitates communication and explanation among all stakeholders including developers of how requirements would be implemented.
 - Highlights early design decisions that will have a profound impact on all software engineering work that follows.
 - Constitutes a relatively small model/blueprint/prototype of how the system components are structured and work together

Introduction to Software Architecture

- **Why Software Architecture is Important:**

- Provides a representation that facilitates communication and explanation of complex requirements among all stakeholders including

Making good decisions while defining the software architecture is critical to the success of a software product

- software engineering work that follows.
- Constitutes a relatively small model/blueprint of how the system components are structured and work together

Introduction to Software Architecture

- **Who does the Software Architecture :**
 - Software Architect :
 - Designs a **functional** system with the qualities of being **Fast, Secure, Reliable and Maintainable** by selecting the optimal technology platforms and architectural patterns to achieve such goals.
 - Must be familiar with technological tools and concepts
 - Have a good understanding of specification requirements
 - **Must** have coding/programming skills. ? **Why**

Introduction to Software Architecture

- **Who does the Software Architecture :**

- Software Architect :

**Because they can envision whether particular architecture is feasible to implement
not wait for the developers to tell and humiliate the architect after months of implementation that the architecture is not feasible to implement**

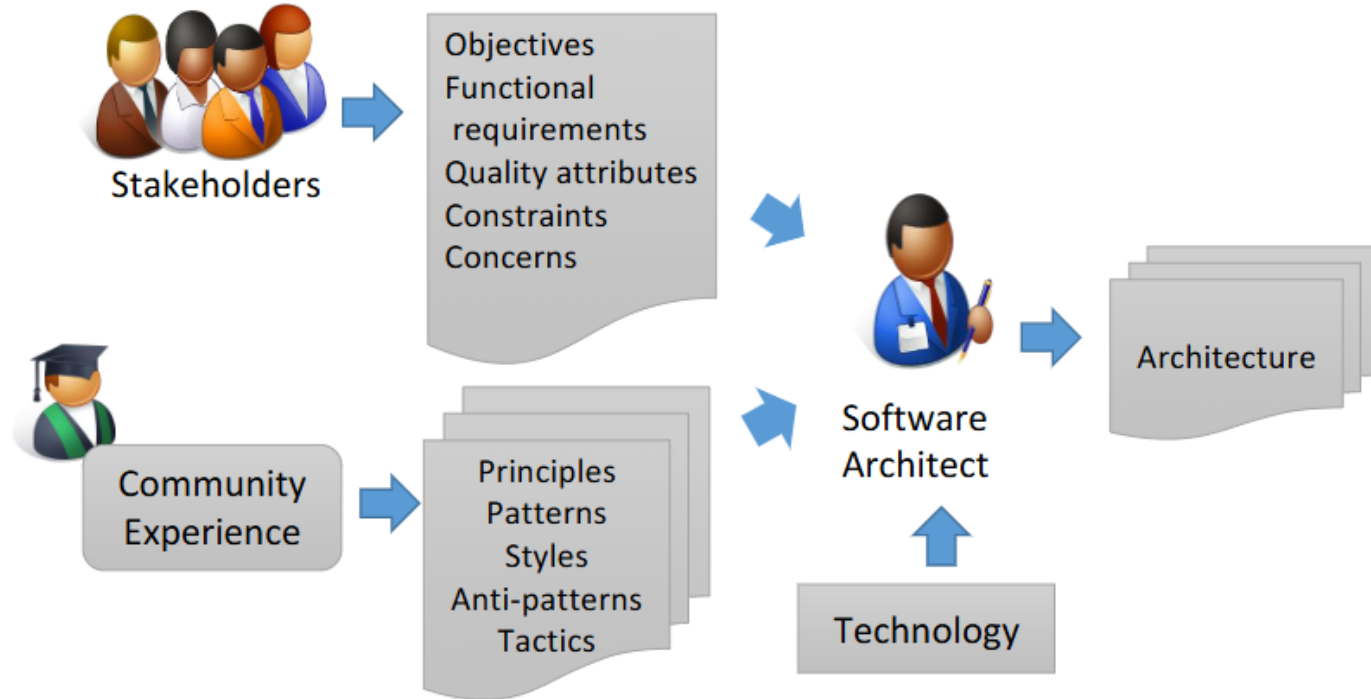
- Have a good understanding of specification requirements
 - **Must** have coding/programming skills. ? **Why**

Introduction to Software Architecture

- **How to be a Software Architect :**

- You need to start your career as a Software Developer
- Have a lot of **experience** in developing different types of software products. * (Estimated no less than 5 years)
- Money wise ?

Role of Software Architect



Software Architecture Process

- **Understand the Functional Requirements**

01

- Define the system goals.
- List the functional use cases and what the system should do.

Software Architecture Process

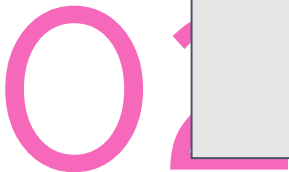
- *Understand the Non-Functional Requirements*

02

- Define Technical and Service Level attributes. This includes:
 - #Users, Performance requirements, volume of data, network capability,,...
- They are **not always known to the business analyst or client**

Software Architecture Process

- *Understand the Non-Functional Requirements*



Non-functional requirements are more important to the architect than functional requirements as they impact heavily the structure of the software

Software Architecture Process

- *Map the Components*

03

- Define the components, modules or subsystems that would constitute the software system.
- Explain the goal, tasks and functionality of each component describing what they would accomplish.

Software Architecture Process

- *Select the Technological Stack*

- Software architects with developers would select :
 - What Platform/Programming Frameworks/Programming Language the system will be based including : backend(s), front-end(s), data stores and communication.
 - Tools Software /APIs/External Services the system would rely on.

04

Software Architecture Process

- *Select the Technological Stack*

It is SUPER IMPORTANT ? WHY

- Tools Software /APIs/External Services the system would rely on.

Software Architecture Process

- *Select the Technological Stack*

- Because :

- It is irreversible : A change in a technology framework or platform may require a complete re-implementation of the software + substantial delay + going over budget..
- Emotional : Developers usually prefer to stick to some technologies they subjectively prefer even though it is inferior in terms of performance.

04

Software Architecture Process

- *Select the Technological Stack*

- What factors to consider when choosing the technology:

- Can perform the task

- Does it run on Mobile devices ?
- Does it run across different platforms ?
- Does it offer a library for GUIs ?
- Does offer AI libraries to process data ?
- Does it support storage of large data ?
- Does it have a libraries for a specific hardware ?

04

Software Architecture Process

- *Select the Technological Stack*

04

- What factors to consider when choosing the technology (cont):
 - Large Active Community and Support
 - Check StackOverflow, See recentness of questions, number of questions from the tags.
 - If you get stuck, at least, there is an active community to assist you or find solution to most common problems.

[Home](#)
[PUBLIC](#)
[Questions](#)
[Tags](#)
[Users](#)
[Companies](#)
[COLLECTIVES](#)
[Explore Collectives](#)
[TEAMS](#)

Stack Overflow for Teams – Start collaborating and sharing organizational knowledge.


[Create a free Team](#)
[Why Teams?](#)

Tags

A tag is a keyword or label that categorizes your question with other, similar questions. Using the right tags makes it easier for others to find and answer your question.

[Show all tag synonyms](#)

[Popular](#)
[Name](#)
[New](#)
[javascript](#)

For questions about programming in ECMAScript (JavaScript/JS) and its different dialects/implementations (except for ActionScript). Note that JavaScript i...

2492567 questions

370 asked today, 2428 this week

[python](#)

Python is a multi-paradigm, dynamically typed, multi-purpose programming language. It is designed to be quick to learn, understand, and use, and enforc...

2132212 questions

576 asked today, 3541 this week

[java](#)

Java is a high-level object-oriented programming language. Use this tag when you're having problems using or understanding the language itself. This...

1896019 questions

203 asked today, 1280 this week

[c#](#)

C# (pronounced "see sharp") is a high-level, statically typed, multi-paradigm programming language developed by Microsoft. C# code usually targets...

1590194 questions

188 asked today, 1086 this week

[php](#)

PHP is an open source, multi-paradigm, dynamically-typed and interpreted scripting language designed initially for server-side web development. Use this...

1458398 questions

102 asked today, 721 this week

[android](#)

Android is Google's mobile operating system, used for programming or developing digital devices (Smartphones, Tablets, Automobiles, TVs, Wear, Glas...

1403621 questions

128 asked today, 854 this week

Software Architecture Process

- *Select the Technological Stack*

04

- What factors to consider when choosing the technology (cont):
 - Popularity : Even though it is superficial. It is important as it can give better insights on performance, portability and richness of features....
 - Google Trend can be used



● **laravel**
Search term

● **django**
Search term

● **nodejs**
Search term



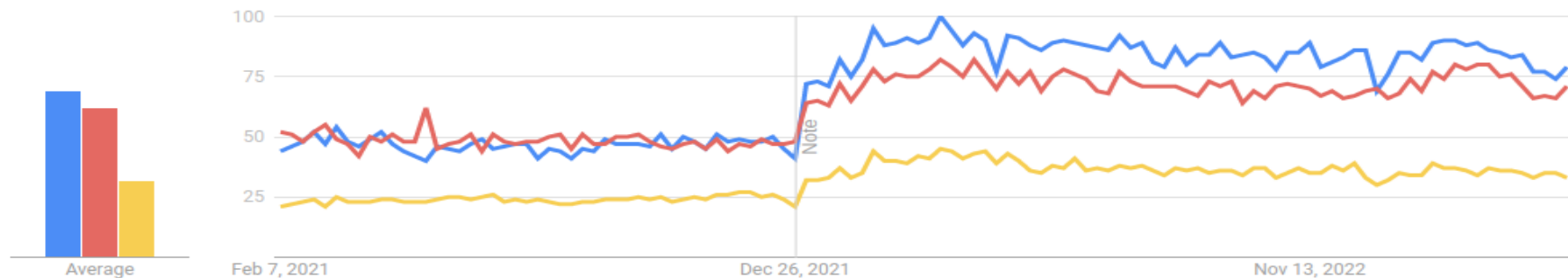
Worldwide ▾

2/4/21 - 5/2/23 ▾

All categories ▾

Web Search ▾

Interest over time ⓘ



Software Architecture Process

- *Select the Technological Stack*
 - What factors to consider when choosing the technology (cont):
 - Licensing and Cost
 - Maintenance and future upgrades
 - Documentation
 - Legal Issues and Regulations

04

Software Architecture Process

- *Select the Technological Stack*

- Backend Technology

- Backend is :

- The part of the software product that can be utilized by the system administrators or product owners/staff or other external systems to **operate, manage and communicate with the** software product.
- Not meant to be accessed by the end-user.

04

Software Architecture Process

- *Select the Technological Stack*
 - Backend Technology
 - We can develop as backend components:
 - Web Applications
 - Web APIs
 - Console-Based System
 - Services

04

Software Architecture Process

- *Select the Technological Stack*

- Backend Technologies

- .NET Classic 

- .NET Core



- Java



- node.js



- PHP



- Python



04

	App Types	Type System	Cross Platform	Community	Performance	Learning Curve
.NET	All	Static	No	Large	OK	Long
.NET Core	Web Apps, Web API, Console, Service	Static	Yes	Medium and growing rapidly	Great	Long
Java	All	Static	Yes	Huge	OK	Long
node.js	Web Apps, Web API	Dynamic	Yes	Large	Great	Medium
PHP	Web Apps, Web API	Dynamic	Yes	Large	OK -	Medium
Python	All	Dynamic	Yes	Huge	OK -	Short

Software Architecture Process

- *Select the Technological Stack*

- Frontend Technology

- Front-end Component is the user interface with which the human end-user interacts.

- We can develop Front-end components for :

- Web Applications
- Desktop Applications
- Mobile Applications
- Console (Not now..)

04

Software Architecture Process

- *Select the Technological Stack*

- Frontend Technology

- Web Applications:

- HTML, CSS, JavaScript.

- There are many frameworks to facilitate the development...

- React, Angular, Bootstrap, Tailwind, JQuery,..

04

Software Architecture Process

- *Select the Technological Stack*
 - Frontend Technology
 - Desktop Applications:
 - Always Look for a rich UI Toolkit
 - Tkinter
 - Electron
 - Swing, JavaFX,,,
 - QT
 - WinForms, WPF, UWP
 - wxWidget
 - ..

04

Software Architecture Process

- *Select the Technological Stack*

- Data Storage Technology

- Consider the requirements:

- Types of transactions ? Need for ACID ?
- Size and type of the data ?
- Reliability requirement ?
- Number of users accessing at the same time ?
- Types of operation : read vs write ?
- Number of operations per minute ?
- Privacy Issue
- ...

04

Software Architecture Process

- *Select the Technological Stack*

- Data Storage Technology

- Database

- Relational SQL-Based DBMS :

- Very mature and enforces a relational schema, but storing large amount of data, can impact the performance.
- MySQL, PostgreSQL, ClickHouse, Oracle, SQLite, HSQLDB...

- NoSQL : Stored as JSON, great emphasis on scale and performance. But, mostly schema-less, no support for transactions and querying can be frustrating. Example, MongoDB

- XML File : Semi-structured text, can be queried using XPath or xQuery.

04

Software Architecture Process

- *Select the Technological Stack*

- Data Storage Technology

- Files:

- Local File System
- Cloud Storage : S3
 - Amazon AWS S3, Azure, Google Cloud, Digital Ocean Spaces...
- Database ?
- CDN
- Distributed Servers..

04

Software Architecture Process

- *Select the Technological Stack*
 - Communication Technology
 - How objects, components, services or subsystems communicate ? What data format being used for the transfer of data ?
 - RPC ? RMI ? CORBA ? REST ? SOAP ? Message-Queue (Kafka, Redis ...)?

04

Software Architecture Process

- *Design the Architecture*

- Based on
 - The functional requirements
 - Non-Functional requirements
 - Technological Stack
- Architects would creatively propose a blueprint describing the structure and behavior of the components

05

Software Architecture Process

- *Write the Architecture Document*

06

- Architects produce an architectural document describing the high-level process to implement the software.
- A good document is relevant for all levels in the organization including CEO, CTO and the development team.

Software Architecture Process

- *Support the Development Team*

- The job for the architect is not done once the software architectural document is done.
- The architect needs to support the team to:
 - Make sure they are implementing according to the architecture.
 - Solve any dilemmas and re-adapt the architecture due to non-anticipated problems and issues.

07

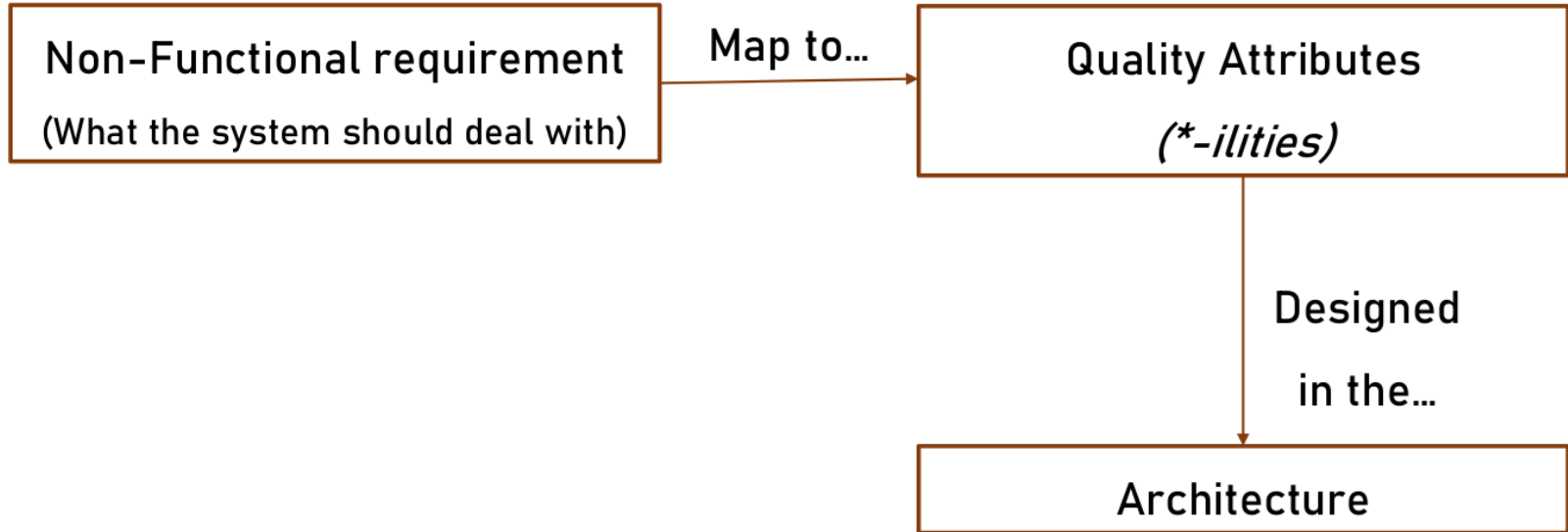
Architecture Quality Attributes

- ***Quality Attributes (*-ilities)***

- Technical capabilities that should be used in order to implement and fulfill the non-functional requirements
- They are closely tied to non-functional requirements
- Example : Non-functional requirement statement :
 - “The to-do App must be implemented to work under heavy load due to many users accessing at the same time”
 - Quality Attribute or Technical capability : **Scalability**

Architecture Quality Attributes

- **Quality Attributes (*-ilities)**



Architecture Quality Attributes

- ***Importance of Non-Functional Requirements :***
 - “The to-do App must be implemented to work under heavy load due to many users accessing at the same time”

Architecture Quality Attributes

- ***Importance of Non-Functional Requirements :***
 - “The to-do App must be implemented to work under heavy load due to many users accessing at the same time”
 - **Vs**
 - “The to-do App must be implemented to work under heavy load due to many users accessing at the same time and not waste money on unused resources”

Architecture Quality Attributes

- *Importance of Non-Functional Requirements :*

- “The  users access
- **Vs**
- “The to-do App must be implemented to work under heavy load due to many users accessing at the same time and not waste money on unused resources”

Architecture Quality Attributes

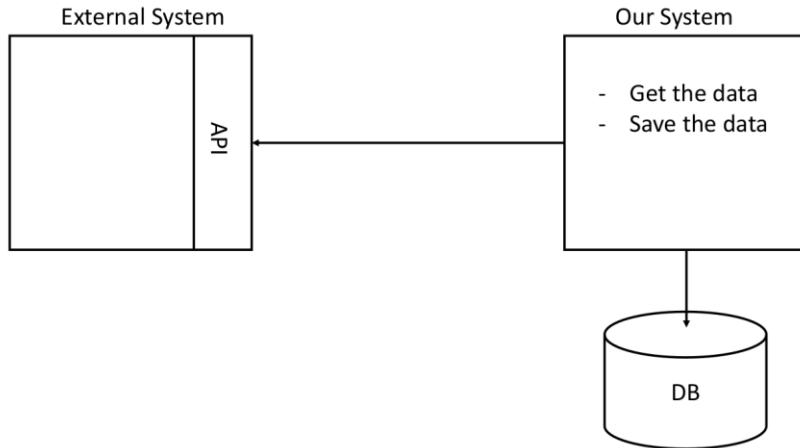
- ***Scalability :***

- The system capacity to handle heavy load due to an increase of :
 - Number of users
 - Size of the data
 - Running or deployed instances.
- Scale Up (Vertical) : Add more processors, RAM, Hard-drives...
- Scale Out : Add more Machines, VMs to the pool of servers.

Architecture Quality Attributes

- **Modularity :**

- Modular system is built from (smaller) building blocks that can be changed or replaced or integrated without affecting the whole system.

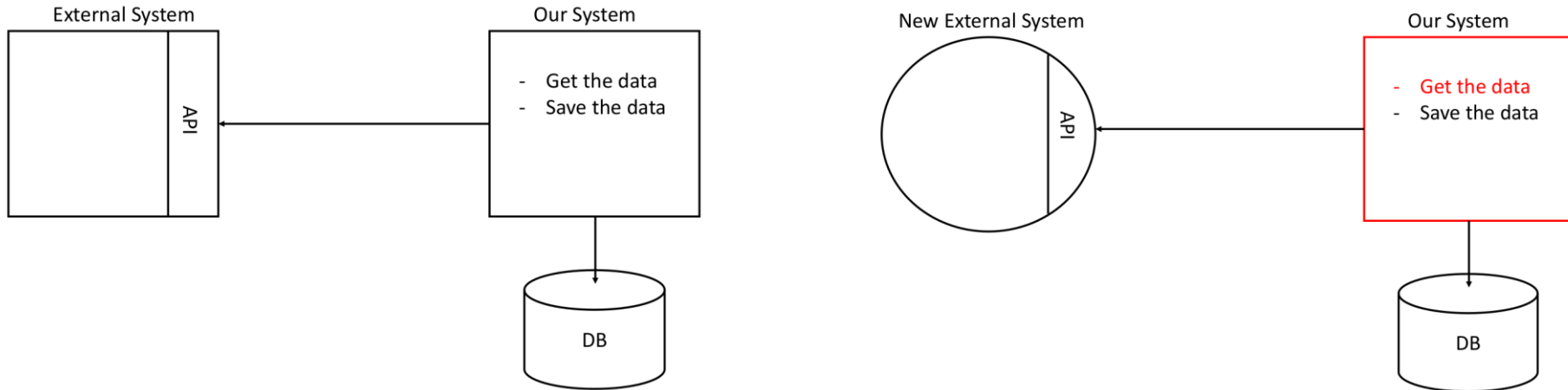


Assume that the API has changed ?

Architecture Quality Attributes

- **Modularity :**

- Modular system is built from (smaller) building blocks that can be changed or replaced or integrated without affecting the whole system.



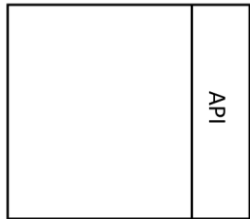
Architecture Quality Attributes

- **Modularity :**

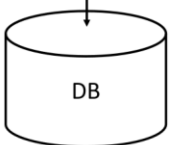
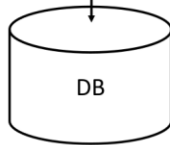
- Mod

- repla

External System



It requires modifying and testing the full system + FULL and long deployment



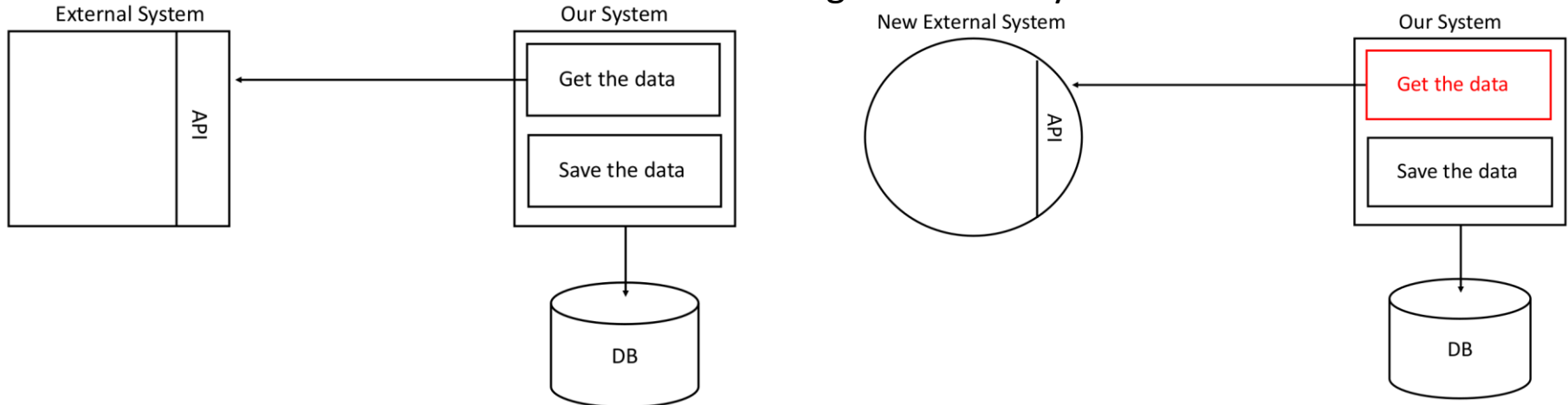
m

data
data

Architecture Quality Attributes

- **Modularity :**

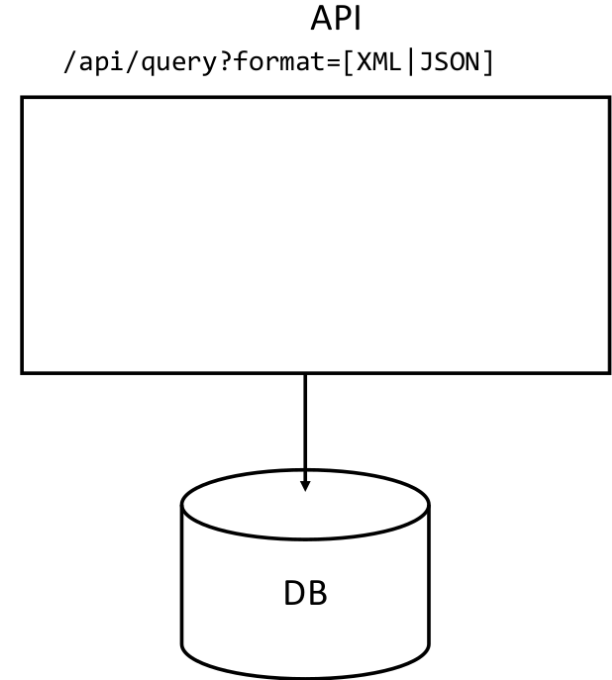
- Modular system is built from (smaller) building blocks that can be changed or replaced or integrated without affecting the whole system.



Architecture Quality Attributes

- **Extensibility :**

- The capability of a system where its functionality can be extended to add new features **without modifying its existing code**
- Example : We have an Backend API service to generate XML and JSON (Two formats)
- **We need to CSV format ?**



API

/api/query?format=[XML | JSON | CSV]

```
switch (format) {  
  case "xml":  
    return formatXml(data);  
  case "json":  
    return formatJson(data);  
}
```

How it can be extended to add CSV ? without modifying the actual code

```
switch (format) {  
  case "xml":  
    return formatXml(data);  
  case "json":  
    return formatJson(data);  
  case "csv":  
    return formatCsv(data);  
}
```

API

/api/query?format=[XML | JSON | CSV]

This is totally the incorrect way of doing it

API

/api/query?format=[XML | JSON | **CSV**]

```
switch (format) {  
    case "xml":  
        return formatter.Format(data);  
}
```

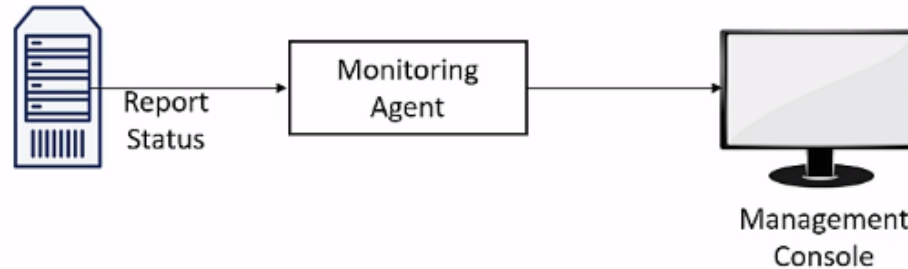
```
String formatQuery(string format,  
                   string data) {  
    IFormatter formatter =  
        GetFormatter(format);  
  
    return formatter.Format(data);  
}
```

We use design patterns.

Architecture Quality Attributes

- ***Manageability :***

- It is the ability to know what's going on with the software application at any given time in order to take actions to improve its performance.



- Self-reporting can help to automate the system to be manageable and improve recovery from failure or fault tolerance (To respond to failure and continue functioning normally)

Architecture Quality Attributes

- ***Other qualities :***

- adaptability
- deployability
- upgradability
- recoverability

....

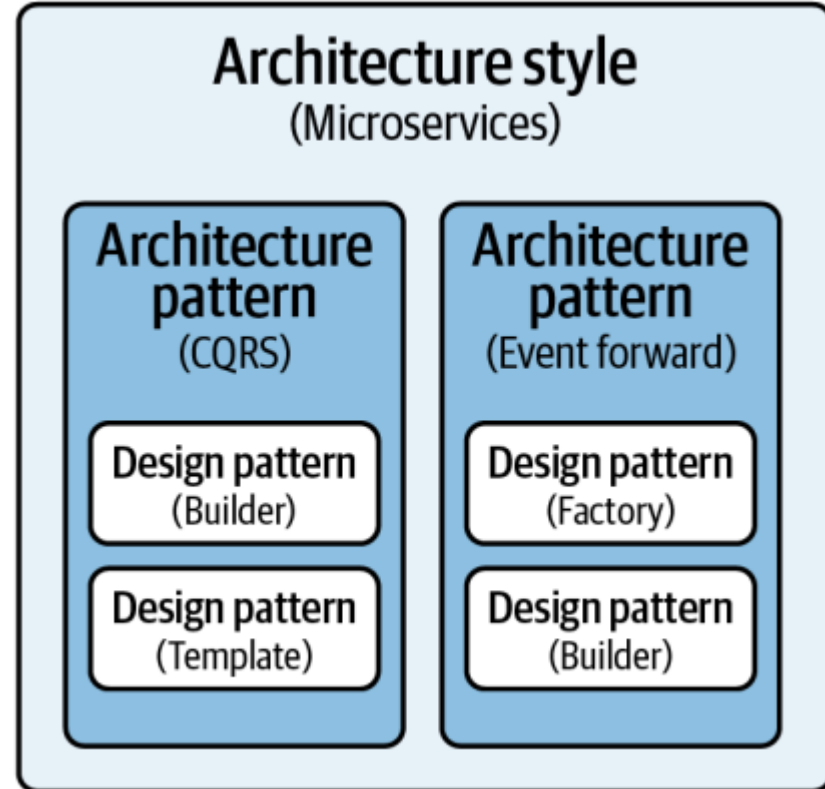
- https://en.wikipedia.org/wiki/List_of_system_quality_attributes

Software Architectural Styles

- ***What's an Architectural Style***
 - Reusable solution, pattern or template to a commonly occurring problem in software architecture within a given context.
 - It facilitates the structuring of components by using a well-tested solution.
 - The use of terms : style vs. pattern is still debatable among software engineers.
 - Architecture Pattern describes reusable structural building blocks.
 - Architecture Style describes the overall system consisting of architectural patterns.

Software Architectural Styles

- ***What's an Architectural Style***
 - Examples :
 - Microservice (Arch. Style)
 - CQRS : Command Query Responsibility Segregation. (Arch. Pattern)
 - Design Pattern : Low-level architectural design, not to be confused with architectural pattern.



Software Architectural Styles

- ***Architecture Classification***

- **Monolithic Architectures :**

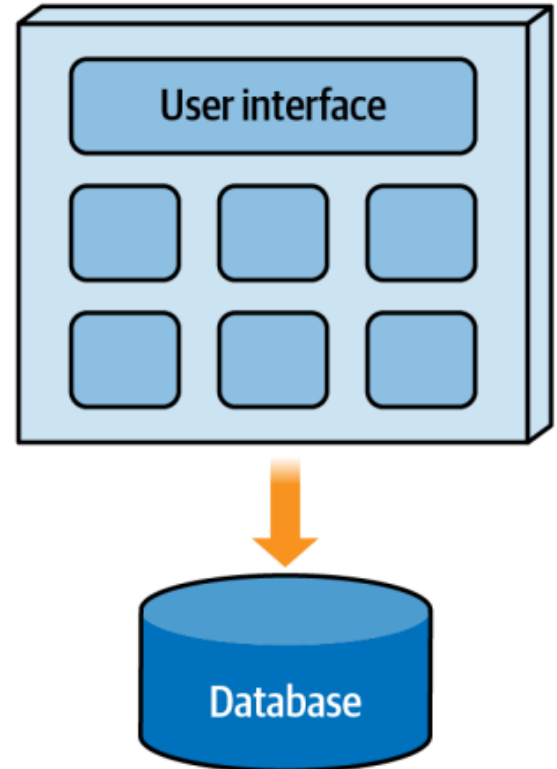
- Single deployment Unit.

- Good Points :

- Must easier to design and implement
- Inexpensive
- Deployed more quickly.

- Weak Point :

- Scalability (Only Vertical – adding more resources
- Fault tolerance



Software Architectural Styles

- **Architecture Classification**

- **Distributed Architectures :**

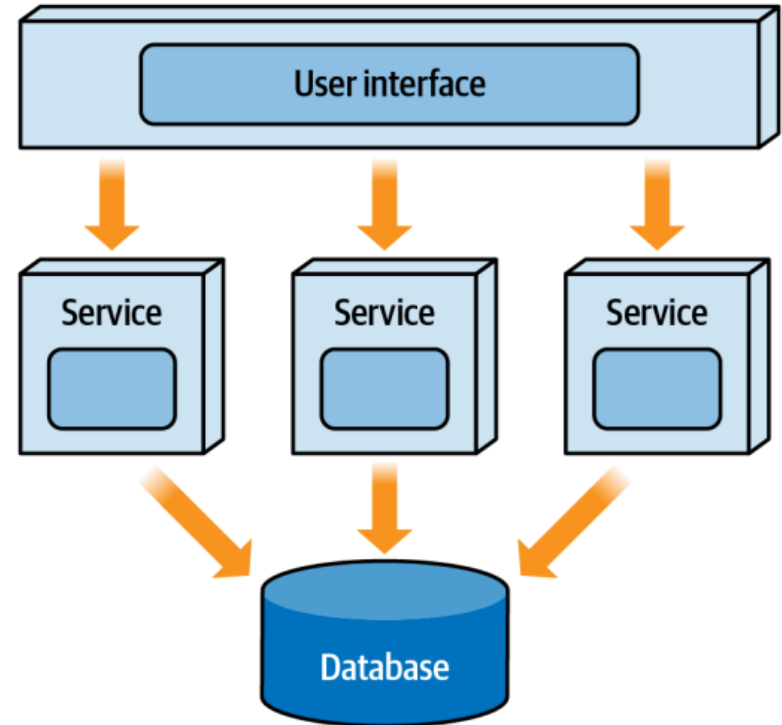
- Multiple deployment Units.

- **Good Points :**

- Scalability (Up or Out)
 - Performance
 - Fault Tolerance

- **Weak Point :**

- Expensive
 - Maintenance
 - Complexity



Software Architectural Styles

- **Architecture Classification**

- **Distributed**

- Multiple

- Good

- S

- P

- P

- Weak

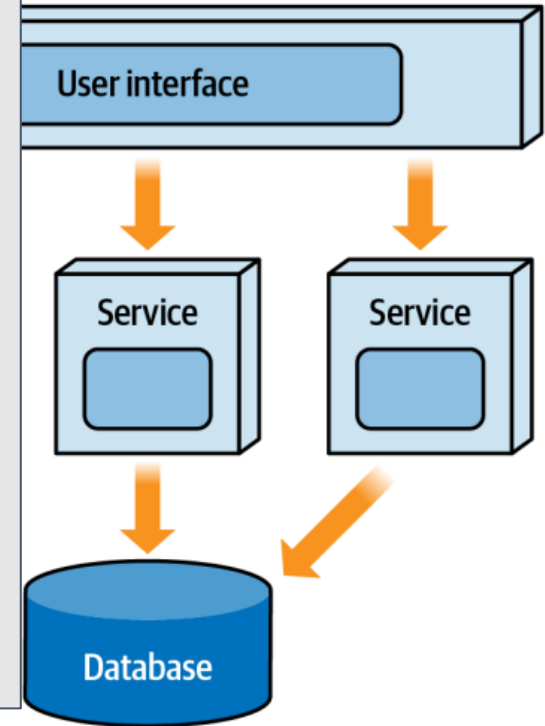
- E

- M

- Complexity

Distributed architectures are plagued with the 8 fallacies of distributed computing :

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. The network is secure
5. Topology doesn't change
6. There is one administrator
7. Transport cost is zero
8. The network is homogeneous.

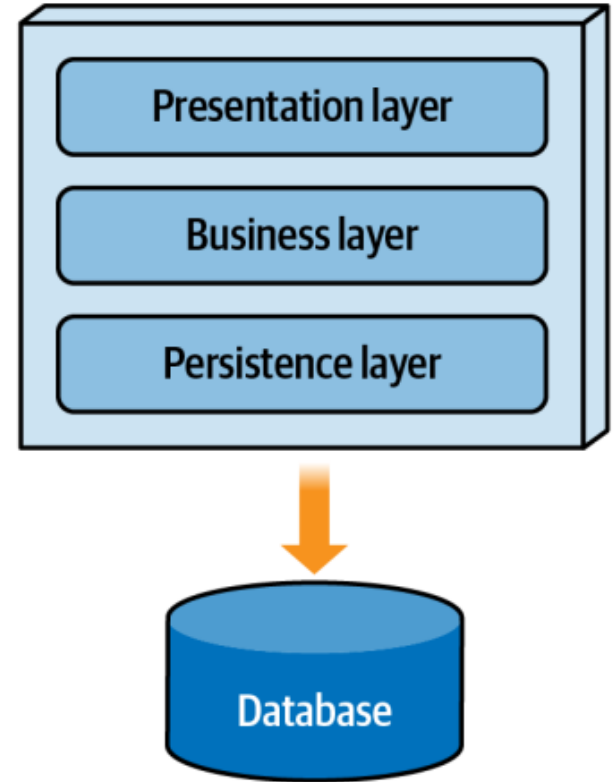


Software Architectural Styles

- ***Architecture Partitioning***

- Architects can design and group functionalities into components using either:

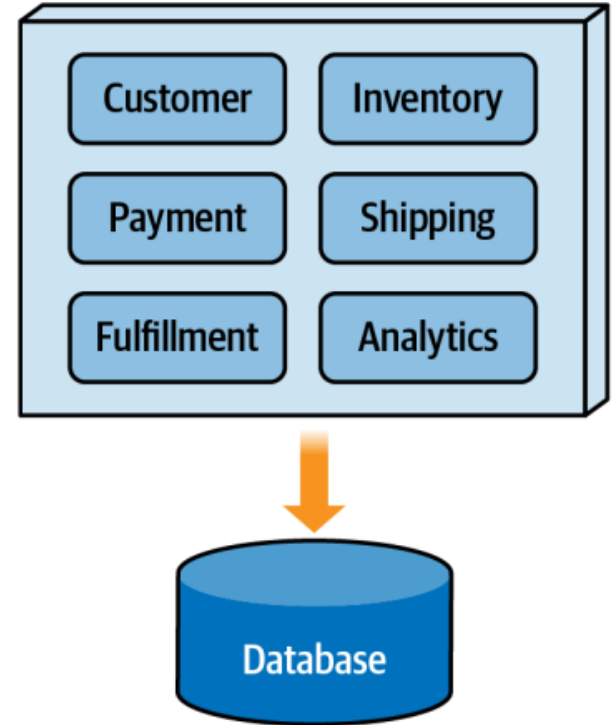
- **Technical Partitioning**
- **Domain Partitioning**



Software Architectural Styles

- ***Architecture Partitioning***

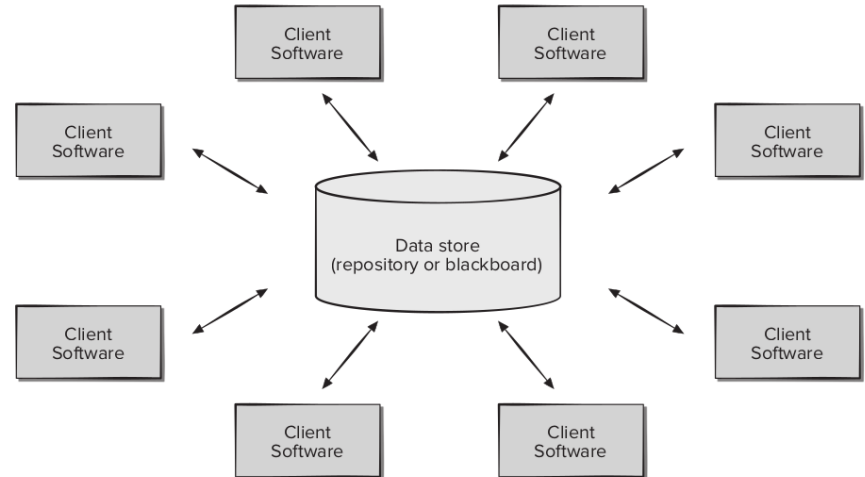
- Architects can design and group functionalities into components using either:
 - Technical Partitioning
 - **Domain Partitioning**



Software Architectural Styles

- ***Data-Centred (Repository) Architecture Style***

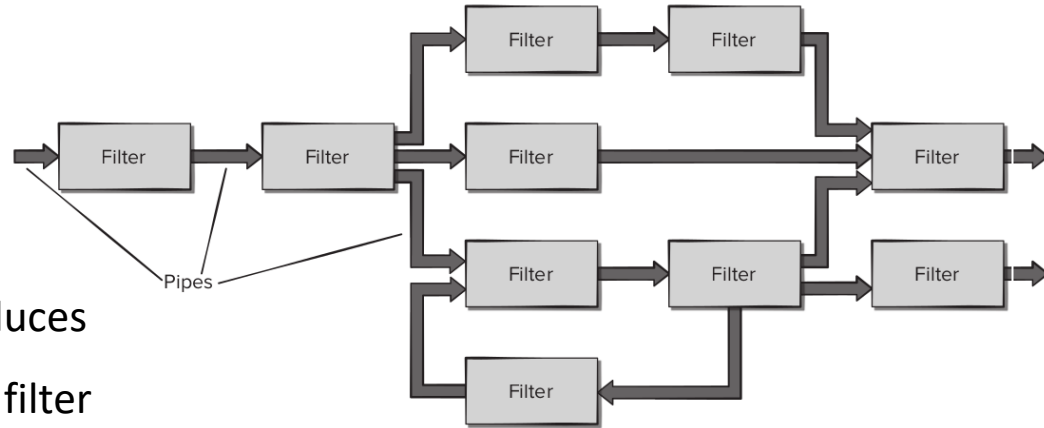
- A data store (e.g., a file or database) resides at the center of this architecture and is accessed frequently by other components that update, add, delete, or otherwise modify data within the store
- A component is not aware of the data or states of other components.



Software Architectural Styles

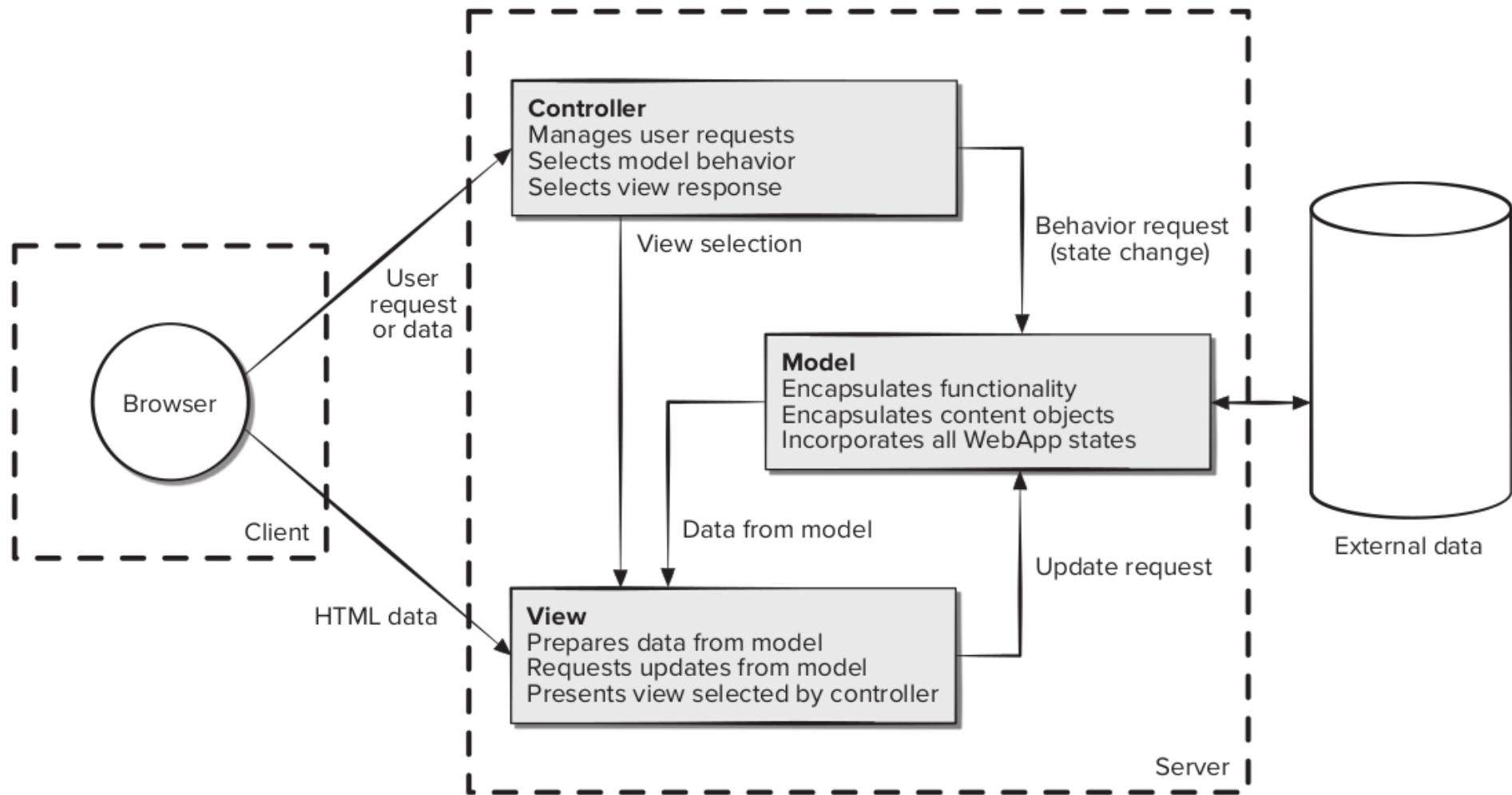
- ***Pipe and Filter (Data-Flow) Architecture Style***

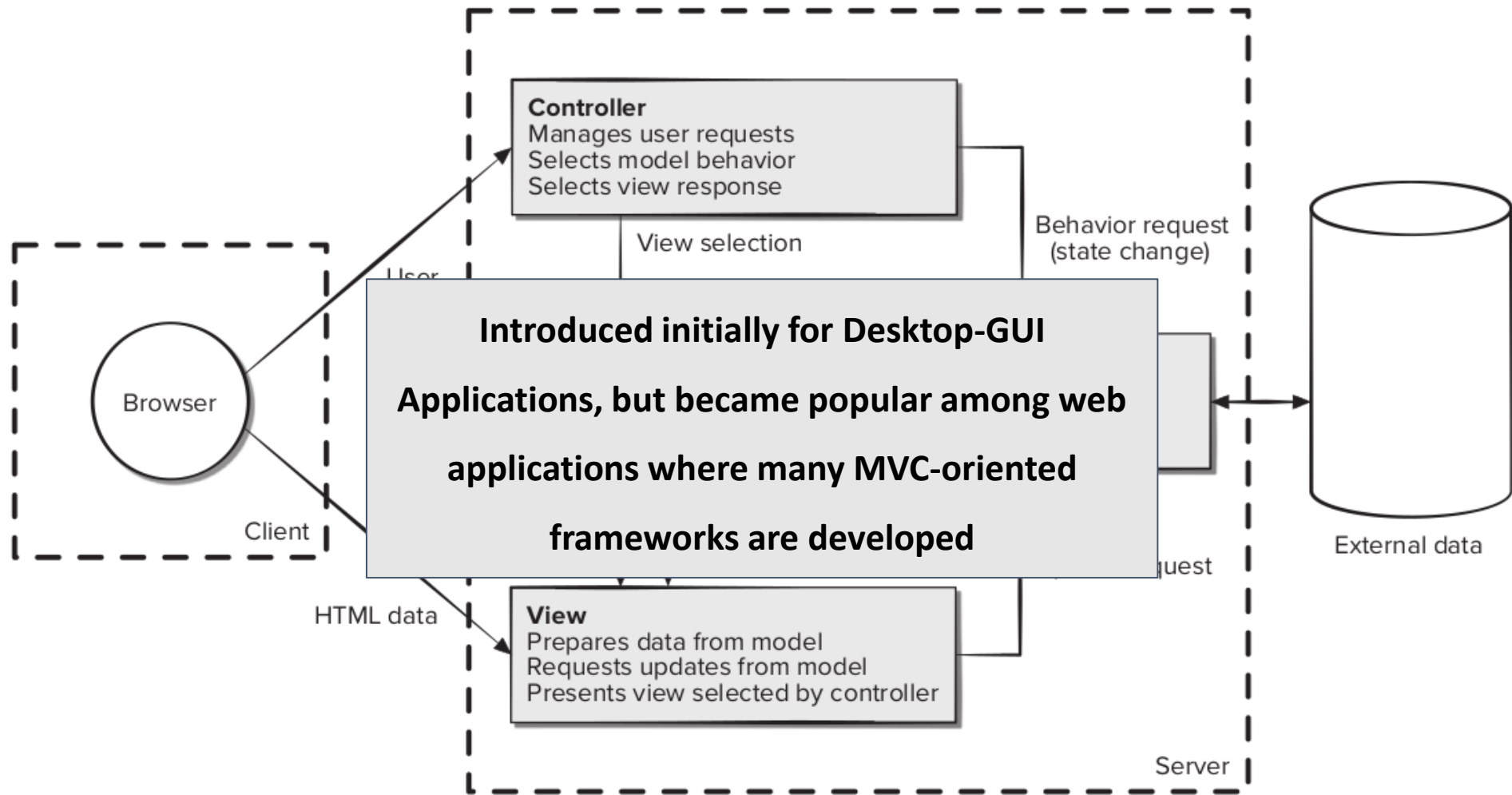
- has a set of components, called filters, connected by pipes that transmit data from one component to the next.
- Each filter works independently of other components
- A Filter expects data from previous Filter, does some process and produces data in a form suitable for the next filter



Software Architectural Styles

- ***Model-View-Controller (MVC) Architecture Style***
 - Composed of three components:
 - Model : contains all application-specific content and processing logic including access to the database.
 - View : contains all interface-specific functions and enables the presentation of content and processing logic required by the end user
 - Controller : manages access to the model and the view and coordinates the flow of data between them





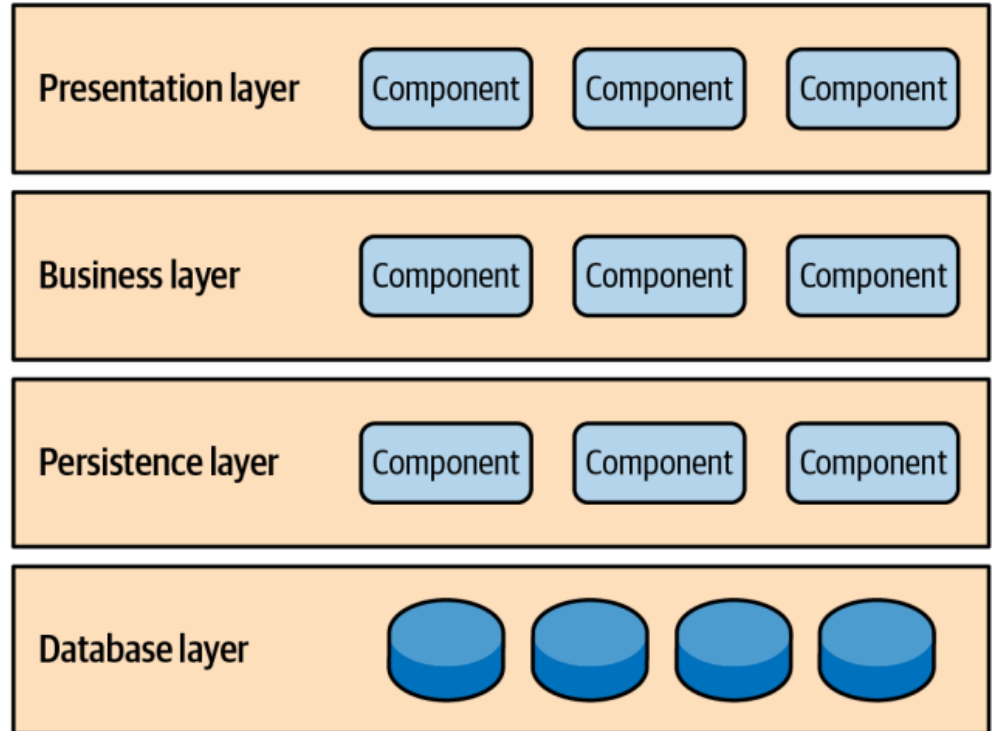
Software Architectural Styles

- ***Layered Architecture Style***

- Also known as the “n-tier architecture” (Debatable)
- Contains a number of layers where each layer has a specific role and responsibility within the application
- Examples of layers include :
 - Presentation : for rendering the user interface
 - Business Logic : for computing, processing requests,..
 - Persistence : for loading, searching and saving to the database
 - database layer : for storing data into the actual DBMS

Software Architectural Styles

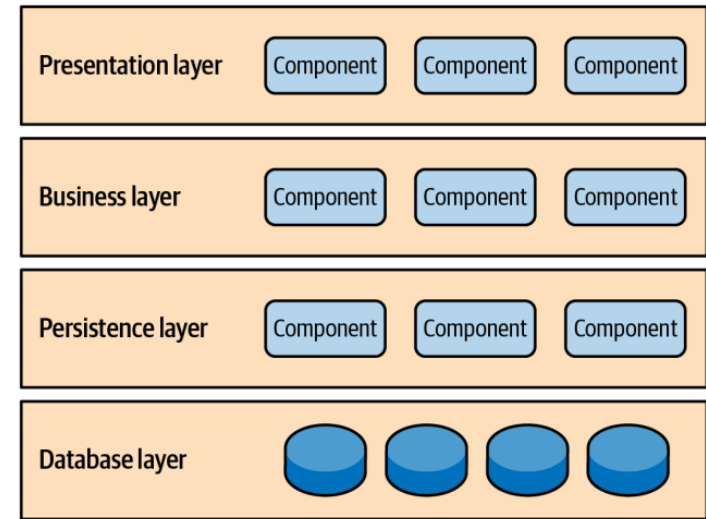
- ***Layered Architecture Style***
 - Can the presentation layer communicate directly with the database ? Faster ? Quick ?
Or can the database layer updates the presentation layer directly ?



Software Architectural Styles

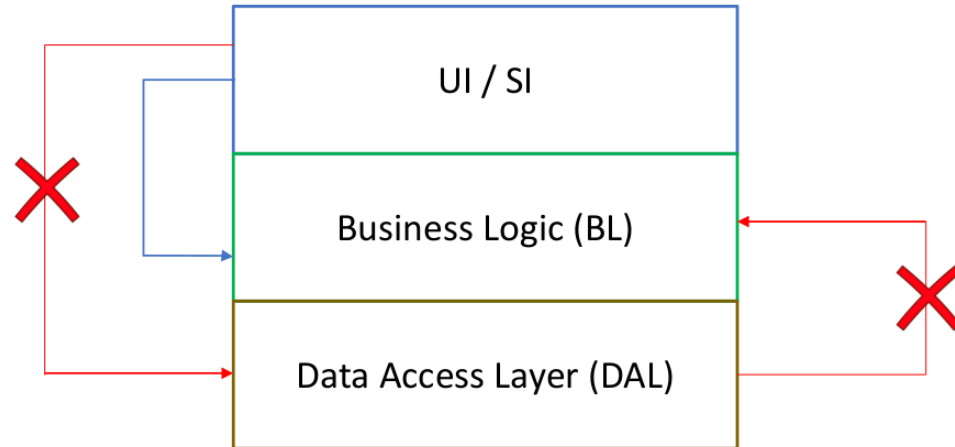
- ***Layered Architecture Style***

- Architecture by default is closed layer to enforce the layers of isolation
- Closed Layer: Only layer adjacent to each other can communicate. (Opposite or open Layered architecture)
- Layers of isolation = “ a change made in one layer don't impact or affect component on other layers”



Software Architectural Styles

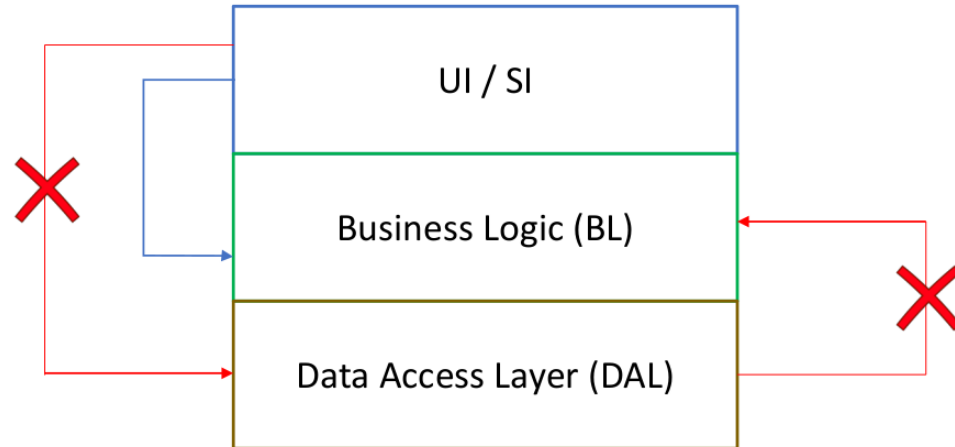
- ***Layered Architecture Style***
 - Separate the module or decouple them as much as possible.
 - If you change the data access layer, you won't need to change the BL or UI.



Software Architectural Styles

Pay a lot of attention to the direction of the arrows, who is calling who ?

- **Layered Architecture Style**
 - Separate the module or decouple them as much as possible.
 - If you change the data access layer, you won't need to change the BL or UI.



Software Architectural Styles

- ***Layered Architecture Style***

- ***When to use :***

- Time or budget constraints : It is simple (and mostly monolithic style) and no complexity is involved. Secondly, most developers are familiar with this style, making it easier to implement.
 - For the case when the changes are mostly isolated to specific layer (Changing the business rules only, or changing the user interface).

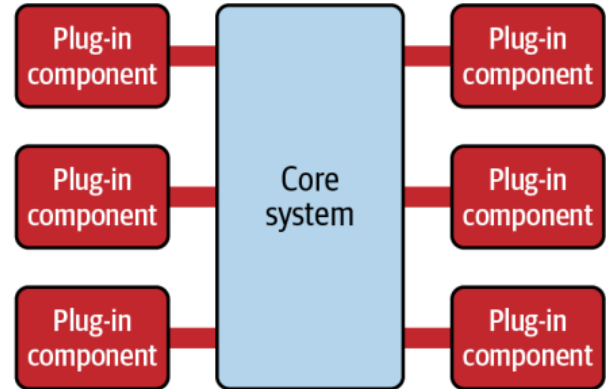
- ***When not to use:***

- For smaller projects
 - When layers are tightly coupled
 - Performance ? vs. Scalability

Software Architectural Styles

- ***Microkernel Architecture Style***

- is a flexible and extensible architecture that allows a developer or end user to easily add additional functionality and features to an existing software system in the form of extensions, or “plug-ins,”
- Also named : “plug-in architecture style”
- Example :
 - Operating System
 - Chrome/ Firefox
 - Eclipse IDE



Software Architectural Styles

- ***Microkernel Architecture Style***

- ***When to use :***

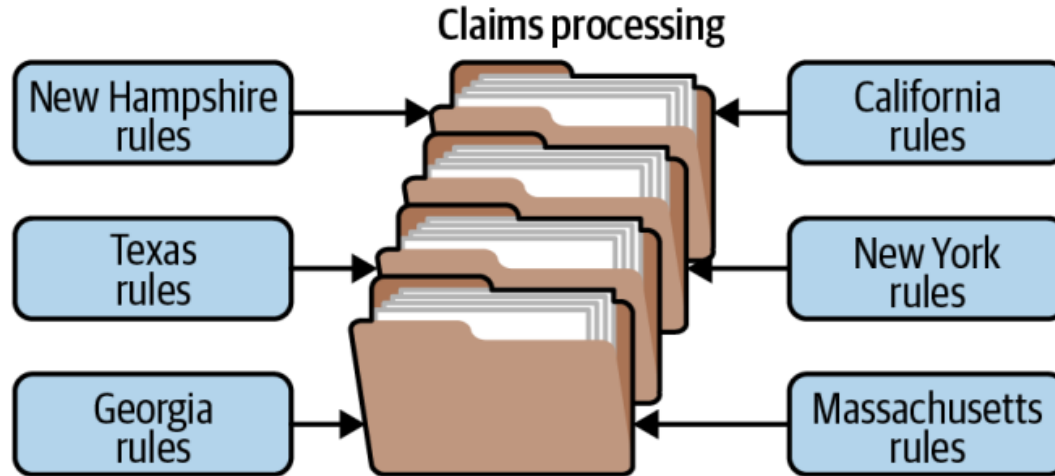
- Product-based application that will have heavily planned extensions
 - Software systems with multiple and/or custom configurations
 - May be we store to different services (cloud, file system..)
 - May be the possibility to with different services.

- ***When not to use:***

- Not scalable as core system acts the main bottleneck.
 - If there will be a change in the core system, it will mess all plugins

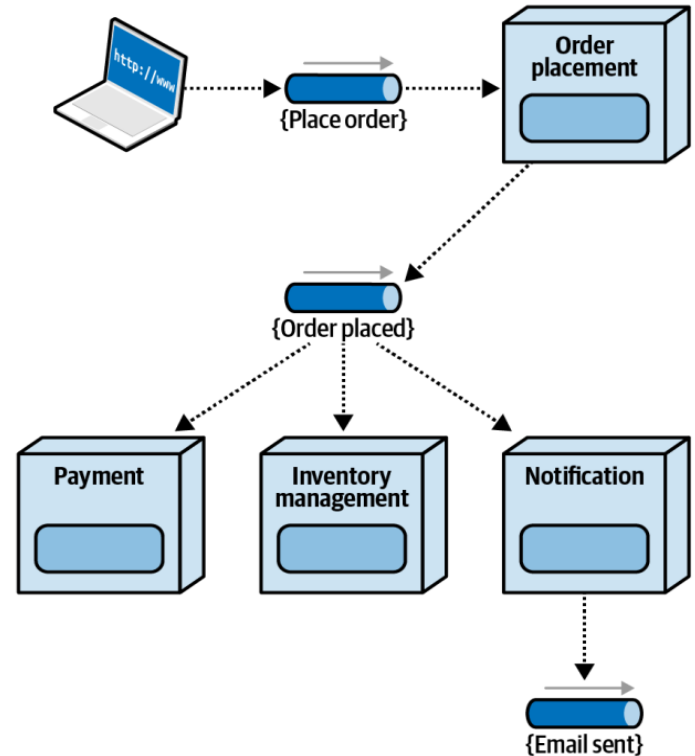
Software Architectural Styles

- *Microkernel Architecture Style*



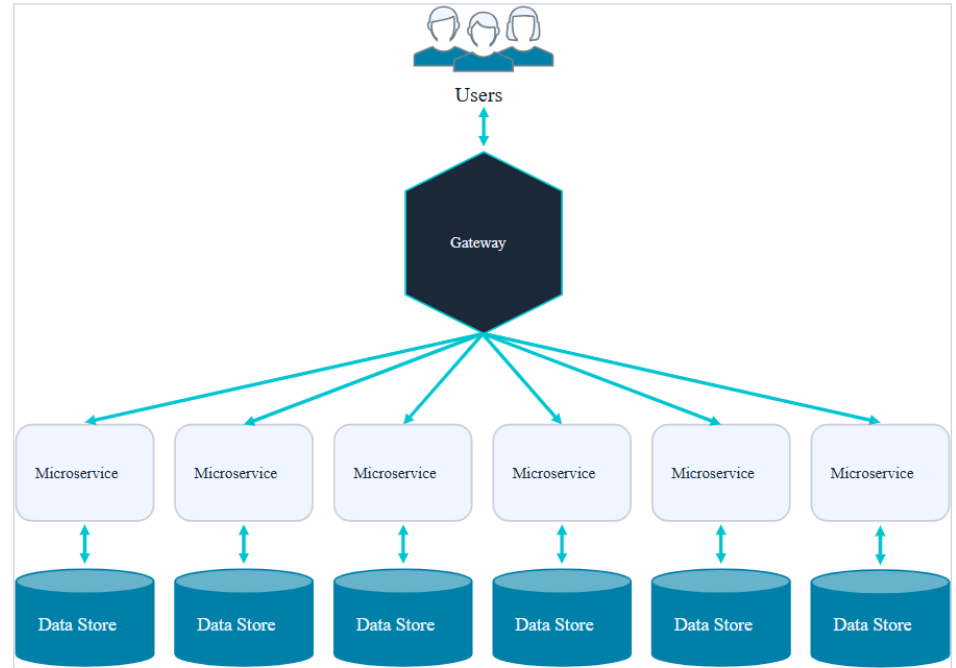
Software Architectural Styles

- ***Event-Driven Architecture Style***
 - Relies on asynchronous processing using highly decoupled event processors that trigger events and correspondingly respond to events happening in the system



Software Architectural Styles

- ***Microservice Architecture Style***
 - A microservice is defined as a single-purpose, separately deployed unit of software that does one thing really, really well



Linus-Tanenbaum Debate (1992)



Linux



Minix

On the Importance of Software Architecture

<https://www.oreilly.com/openbook/opensources/book/appa.html>

The beginning of the debate: Tanenbaum's msg (1992)

From: ast@cs.vu.nl (Andy Tanenbaum)

Newsgroups: comp.os.minix

Subject: **LINUX is obsolete**

Date: 29 Jan 92 12:12:50 GMT

I was in the U.S. for a couple of weeks, so I haven't commented much on LINUX (not that I would have said much had I been around), but for what it is worth, I have a couple of comments now.

Tanenbaum's Argument

- Linux has a monolithic architecture:
 - The entire OS is a single executable file
 - Includes process management, memory, file system, etc.
- Microkernel architecture is better:
 - Kernel contains only essential services
 - Other services run as independent processes

Linus's Reply

From: torvalds@klaava.Helsinki.FI (Linus Benedict Torvalds)

Subject: Re: LINUX is obsolete

Date: 29 Jan 92 23:14:26 GMT

Organization: University of Helsinki

Well, with a subject like this, **I'm afraid I'll have to reply.**

Linus's Argument

- In theory, microkernel architecture is superior
- However, other criteria have to be considered
- For example, Linux is a working reality, not just a promise

A New Response from Tanenbaum

I still maintain the point that designing a monolithic kernel in 1991 is a fundamental error. Be thankful you are not my student. **You would not get a high grade for such a design :-)**

Comment by Ken Thompson (Unix co-creator)

From: kt4@prism.gatech.EDU (Ken Thompson)

Subject: Re: LINUX is obsolete

Date: 3 Feb 92 23:07:54 GMT

Organization: Georgia Institute of Technology

I would generally agree that microkernels are probably the wave of the future. However, it is in my opinion easier to implement a monolithic kernel. It is also easier for it to turn into a mess in a hurry as it is modified.

Ken Thompson predicted the future: 17 years later (2009),
Torvalds stated at a Linux conference...

"We are definitely not the streamlined, small, hyper-efficient kernel that I envisioned 15 years ago. The kernel is huge and bloated... And whenever we add a new feature, it only gets worse."

Is Linux kernel getting bloated ? Linus
Torvalds says Yes!

September 24, 2009 Posted by Ravi

Key takeaway: The costs of architectural decisions may take years to become apparent...

Credit:

- Prof. Imed Boucherika, Software Engineering course, ENSIA 2024/2025